Algorithms (Tree)

作者：羅浚嘉

單位： 國立澎湖科技大學 資工系2甲

學號:1113405040

開發環境： C++

Email:d13405040@ems.npu.edu.tw

摘要 (Abstract)

本實驗旨在利用 C++ 程式語言實作基礎資料結構與演算法之整合應用。透過線性結構（Stack & Queue）處理基礎數據，並進一步建構二元樹結構以進行深度優先遍歷分析。此外，本研究亦導入分治法（Divide and Conquer）邏輯，實作合併排序與二元搜尋，驗證程式在處理大規模數據時的效率與準確性。

程式碼在:https://github.com/d13405040-boop/1113405040

I. 前言 (INTRODUCTION)

在演算法領域中，「分治法」與「樹狀結構」是處理複雜問題的核心工具。透過遞迴邏輯，我們可以將問題拆解為較小的子問題。本實驗透過 C++ 實作，從最基礎的 LIFO/FIFO 概念出發，延伸至動態建立完全二元樹（Complete Binary Tree），並利用遍歷演算法觀察資料在不同邏輯下的輸出規律。

II. 方法與實驗結果 (METHOD AND EXPERIMENTAL RESULTS)

1. 資料結構與樹狀建構

- 線性存取：透過標準函式庫驗證數據在堆疊與佇列中的存取順序。
- 動態建樹：使用遞迴函式根據陣列索引自動生成子節點。若父節點索引為 i，則其左子節點為 2i+1，右子節點為 2i+2。

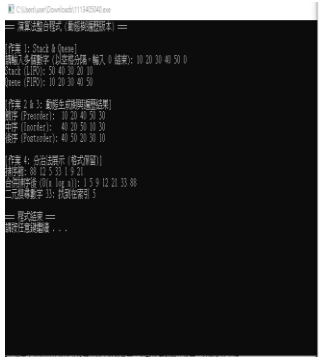
2. 遍歷演算法實作

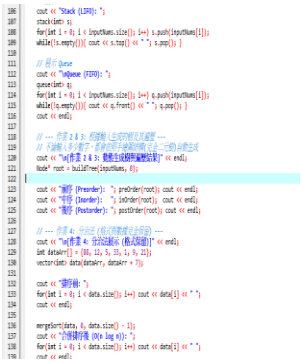
程式實作了三種深度優先搜尋方式：

- 前序遍歷 (Pre-order)：根節點 -> 左子樹 -> 右子樹。
- 中序遍歷 (In-order)：左子樹 -> 根節點 -> 右子樹。
- 後序遍歷 (Post-order)：左子樹 -> 右子樹 -> 根節點。

3. 分治法展示

- 合併排序 (Merge Sort)：將陣列不斷對半切分後再進行合併排序，時間複雜度為 O(n log n)。
- 二元搜尋 (Binary Search)：在已排序的資料中，每次排除一半的搜尋範圍，確保了高效率的檢索速度。





III. 結論 (CONCLUSION)

結構關聯性：實驗證明，透過遞迴邏輯建構的二元樹，其遍歷結果與預期之理論模型完全吻合。

演算法效率：分治法在排序與搜尋任務中表現優異，相較於傳統線性搜尋，大幅提升了程式執行效能。

實務心得：本實驗強化了對於記憶體指標運作與遞迴數學原理的理解，為後續處理更複雜的資訊系統與影像處理演算法奠定了良好的基礎。